

Trabalho Final — A Jornada de Melhoria da Recuperação em RAG

Contents

Trabalho Final — A Jornada de Melhoria da Recuperação em RAG	1
1. Objetivo	1
2. Entregáveis	2
3. O ponto de partida (o que o projeto final já faz)	2
4. As métricas (o que e por que medir)	3
5. Construindo o dataset de avaliação (gabarito)	3
6. Como medir de forma reprodutível	4
7. A jornada (fases dos experimentos)	5
8. Estrutura do relatório (entregável principal)	6
9. Rubrica de avaliação (100 pts)	7
10. Armadilhas comuns (evite perder pontos)	7
11. Cronograma sugerido	8
12. Checklist final	8

Trabalho Final — A Jornada de Melhoria da Recuperação em RAG

Disciplina: RAG & CAG Aplicados a Direito e Segurança Pública **Ponto de partida:** a aplicação do **Projeto Final** (aula12/projeto_final/) **Formato de entrega:** relatório técnico (PDF ou DOCX) + código modificado + dataset de avaliação + planilha de resultados

1. Objetivo

Partindo da aplicação do Projeto Final (uma API de ingestão inteligente + RAG, com OpenSearch, LightRAG, Groq/LLM agnóstico e interface Gradio), você vai **escolher uma fonte de dados própria** (PDF, planilha, DOCX, conjunto de documentos...), **medir a qualidade da recuperação** e conduzir uma **jornada de experimentos** para melhorá-la, aplicando — de forma incremental e medida — as técnicas vistas ao longo do curso.

O entregável central é um **relatório dos experimentos**: o que você mediu, o que mudou no código, qual hipótese testou, e quanto cada técnica melhorou (ou piorou) as

métricas. A nota vem da **qualidade da jornada e da análise**, não de “acertar” um número.

Princípio do trabalho: **medir antes de otimizar**. Toda mudança precisa de um *antes* e um *depois* com as mesmas métricas e o mesmo dataset.

2. Entregáveis

1. **Relatório técnico** (8–20 páginas) seguindo a estrutura da Seção 8.
2. **Código modificado** do projeto final (fork/cópia da pasta aula12/projeto_final/), com as mudanças de cada fase versionadas (commits ou cópias nomeadas: exp01_baseline, exp02_chunk_recurso, ...).
3. **Dataset de avaliação** (avaliacao/dataset.json) — perguntas + gabarito (Seção 5).
4. **Planilha de resultados** (avaliacao/resultados.csv ou .xlsx) — uma linha por experimento (template na Seção 7).
5. **Gráficos** comparando os experimentos (barras por métrica e/ou linha da evolução).

3. O ponto de partida (o que o projeto final já faz)

Antes de mexer, entenda a **baseline** que você recebe:

Etapa	Onde está	Comportamento atual (baseline)
Extração	app/extracao.py	Agente LLM escolhe extrair_planilha / extrair_texto / extrair_com_ocr (Docling); fallback heurístico + PyMuPDF
Decisão de destino	app/indexacao.py → decidir_destino	OpenSearch (padrão) ou LightRAG/grafos (texto longo e rico em entidades)
Chunking	app/indexacao.py → avaliar_chunking / chunkar	5 técnicas (fixo, recursivo, sentence-window, semântico, hierárquico) + tabela
Embedding	app/config.py → config_ollama	Ollama
Busca	app/busca_avancada.py	nomadic-embed-text (768d) baseline, multi_query, rag_fusion, step_back
Geração	app/consulta.py + app/prompts.py	Groq/LLM agnóstico; prompts editáveis

Etapa	Onde está	Comportamento atual (baseline)
Observabilidade	LangfuseConnector	trace por busca (se configurado)

Sua jornada vai **variar uma coisa de cada vez** nesses pontos e medir o efeito.

4. As métricas (o que e por que medir)

Há **duas famílias** de métricas, porque RAG tem duas etapas que falham por motivos diferentes:

4.1 Recuperação / ranking (a busca trouxe os documentos certos?)

Precisam de um **gabarito de relevância** (quais documentos/chunks são relevantes para cada pergunta).

- **Hit@K** — a pergunta teve **pelo menos um** documento relevante no top-K? (0/1, média sobre as perguntas). Mede *cobertura grosseira*.
- **Recall@K** — **que fração** dos relevantes apareceu no top-K. Mede *cobertura fina*.
- **MRR** (Mean Reciprocal Rank) — posição do **primeiro** relevante (1/posição). Mede se o relevante vem **no topo**.
- **NDCG@K** — qualidade do **ranking** ponderando a posição e a **nota** de relevância (gabarito graduado). É a métrica-síntese de ranking.

Você já tem implementação dessas métricas em `bench_embeddings/app/metricas.py` (numpy + scikit-learn). Reaproveite.

4.2 Geração (a resposta final é boa e fiel ao contexto?) — RAGAS

- **Faithfulness** — a resposta é sustentada pelos trechos recuperados (não “alucina”).
- **Answer Relevancy** — a resposta responde à pergunta.
- **Context Precision / Context Recall** — o contexto recuperado é relevante / contém o necessário (este liga a recuperação à geração).

Padrão de RAGAS com juiz Groq já existe nas Aulas 5 e 8. **Atenção:** com Groq use `ResponseRelevancy(strictness=1)` (a Groq só aceita `n=1`).

Regra de ouro: otimize a **recuperação** primeiro (Hit/Recall/MRR/NDCG). Resposta boa em cima de contexto ruim é sorte; resposta boa em cima de contexto bom é engenharia.

5. Construindo o dataset de avaliação (gabarito)

Sem gabarito não há medição. Monte `avaliacao/dataset.json` com **15-30 perguntas** sobre a SUA fonte:

```
{
  "documentos": [{ "id": "D01", "texto": "..."}, ...],
  "queries_benchmark": [
    {
      "id": "Q01",
      "query": "pergunta como um usuário real faria",
      "relevancia": { "D05": 2, "D06": 1},
      "resposta_referencia": "resposta correta e curta (para o RAGAS)"
    }
  ]
}
```

- relevancia = {doc_id: nota} (2 = muito relevante, 1 = relevante) → alimenta NDCG graduado. Pode ser lista ["D05", "D06"] (tudo nota 1).
- resposta_referencia → usada pelo RAGAS (Context Recall / Answer Correctness).

Como criar sem viés (lições das Aulas 5-7): - Escreva perguntas **coloquiais**, como um leigo faria — **não** copie termos/números exatos do documento (senão a baseline já acha tudo e nenhuma técnica “melhora”). O ganho das técnicas aparece quando há **descasamento de vocabulário** entre pergunta e texto. - Inclua perguntas de **3 tipos**: factual de 1 documento; reformulável (boa para multi-query/step-back); e temática/multi-hop (vários documentos — boa para RAG-Fusion e grafo). - Pode usar o LLM para **rascunhar** perguntas a partir dos documentos, mas **revise manualmente** o gabarito (quem decide o que é relevante é você).

6. Como medir de forma reprodutível

Crie uma pasta avaliacao/ com dois scripts (reaproveitando o que já existe):

avaliar_recuperacao.py — para cada pergunta, chama a busca do projeto (uma técnica/config por vez), pega os IDs recuperados e calcula Hit@K, Recall@K, MRR, NDCG@K contra o gabarito. Esqueleto:

```
# reusa a logica de bench_embeddings/app/metricas.py
from app import busca_avancada # ou chama a API POST /consulta
gold = carregar_dataset("avaliacao/dataset.json")
for q in gold["queries_benchmark"]:
    pipe, inputs, chave = busca_avancada.construir(TECNICA, TOP_K, q["query"])
    docs = pipe.run(inputs, include_outputs_from={chave})[chave]["documents"]
    ids = [d.meta.get("id_original") or d.id for d in docs]
    # -> hit@k, recall@k, mrr, ndcg@k usando q["relevancia"]
salvar_linha_csv("avaliacao/resultados.csv", experimento, metricas)
```

avaliar_ragas.py — roda o RAG completo (busca + geração) e calcula as métricas RAGAS (padrão das Aulas 5/8: Faithfulness, ResponseRelevancy(strictness=1), LLM-ContextPrecisionWithReference, LLMContextRecall).

Dica: registre **tudo no LangFuse** (já integrado). Os traces ajudam a explicar *por que* uma técnica ganhou.

Fixe o que não está sendo testado (mesmo top_k, mesmo modelo, mesmo dataset) para o experimento ser **controlado**.

7. A jornada (fases dos experimentos)

Cada fase é uma **hipótese** → **mudança no código** → **medição** → **comparação com a melhor config anterior**. Registre cada uma como uma linha na planilha:

Template da planilha (resultados.csv):

exp	fase	mudança	Hit@5	Recall@5	MRR	NDCG@1	AUC	ASR@1	ASR@5	ASR@10	ASR@20	ASR@50	ASR@100	Observação
-----	------	---------	-------	----------	-----	--------	-----	-------	-------	--------	--------	--------	---------	------------

Fase 0 — Baseline (obrigatória)

Rode o projeto **como veio**: chunking=auto, embedding nomic-embed-text, busca baseline, top_k=5. **Meça e registre**. Tudo daqui pra frente é comparado com isto.

Fase 1 — Ingestão / extração (a qualidade entra aqui)

Hipótese: texto mal extraído limita tudo a jusante. - Compare extrair_texto (Docling) vs fallback PyMuPDF; ligue OCR (extrair_com_ocr) se houver páginas escaneadas/figuras (app/extracao.py). - Para planilhas, confira se extrair_planilha preserva as tabelas (markdown). - *Mede*: inspecione amostras do texto extraído + rode as métricas (extração ruim derruba Recall).

Fase 2 — Chunking (tamanho e estratégia)

Hipótese: a granularidade do chunk muda o que é recuperável. - Force cada técnica em app/indexacao.py / parâmetro chunking: fixo, recursivo, sentenca_janela, semantico, hierarquico. - Varie split_length / split_overlap (em chunkar). Reindexe a cada mudança. - *Mede*: Hit/Recall/NDCG por estratégia. (Aula 2 e Aula 6.)

Fase 3 — Modelo de embedding

Hipótese: o modelo de embedding define o teto da busca densa. - Compare nomic-embed-text (768) vs bge-m3 (1024) vs mxbai-embed-large no Ollama (EMBEDDING_MODEL no .env; ajuste a dimensão). - **Use a bancada bench_embeddings/** para escolher o melhor para o SEU corpus **antes** de reindexar. (Ferramenta do curso.) - *Mede*: a própria bancada dá Hit/Recall/MRR/NDCG/AUC por modelo.

Fase 4 — Recuperação base (top_k e busca híbrida)

Hipótese: combinar léxico (BM25) + denso melhora cobertura. - Varie top_k. - Implemente **busca híbrida BM25 + densa com RRF** (Aula 4: OpenSearchHybridRetriever/

`DocumentJoiner(join_mode="reciprocal_rank_fusion"))`. Acrescente como uma técnica nova em `app/busca_avancada.py`. - (Opcional) **Contextual Retrieval** (Aula 4): enriquecer cada chunk com um resumo de contexto antes de indexar. - *Mede*: densa vs híbrida.

Fase 5 — Query enhancement (já no projeto)

Hipótese: reescrever a pergunta cobre o descasamento de vocabulário. - Compare base-line vs multi_query vs rag_fusion vs step_back (parâmetro técnica). - Ajuste os **prompts** na aba **Configurações** (Gradio) e veja o efeito — registre qual versão do prompt usou. (Aula 7.) - *Mede*: especialmente nas perguntas coloquiais/temáticas.

Fase 6 — Reranking

Hipótese: reordenar os top-N com um cross-encoder sobe o relevante. - Acrescente um **reranker** após a recuperação: `TransformersSimilarityRanker(model="BAAI/bge-reranker-v2-m3")` (Aula 3). Recupere top-20 → reranqueie → top-5. - *Mede*: MRR e NDCG são os que mais devem subir.

Fase 7 — Técnicas avançadas (escolha ao menos 1)

Hipótese: casos difíceis pedem estratégias específicas. - **HyDE** (Aula 6): gerar documento hipotético e buscar por ele. - **Parent-Child / Auto-Merging** ou **RAPTOR** (Aula 6): recuperar trecho pequeno, devolver contexto maior. - **CRAG / Self-RAG** (Aula 8): avaliar a recuperação e corrigir (fallback web/refinamento). - **Grafo (LightRAG)** (Aula 9): para perguntas **multi-hop**, force destino=grafo e compare com OpenSearch. - **Time-aware / Compressão / ColBERT** (Aula 11) se fizer sentido para a sua fonte. - *Mede*: nas perguntas em que a baseline falhou.

Fase 8 — Avaliação da geração (RAGAS)

Hipótese: melhor contexto → resposta mais fiel. - Rode `avaliar_ragas.py` na baseline e na **melhor configuração** de recuperação. - *Mede*: Faithfulness, Answer Relevancy, Context Precision/Recall. (Aulas 5 e 8.)

8. Estrutura do relatório (entregável principal)

1. **Capa** — nome, fonte de dados escolhida e por quê.
2. **A fonte e o dataset de avaliação** — descrição do corpus, como montou as perguntas e o gabarito, quantas perguntas de cada tipo.
3. **Metodologia** — métricas usadas, top_k, como garantiu experimentos controlados, ferramentas (bench_embeddings, RAGAS, LangFuse).
4. **Baseline (Fase 0)** — números de partida.
5. **Experimentos** — uma subseção por fase: hipótese → o que mudou no código (cite arquivo/função) → resultado (tabela) → análise (por que melhorou/piorou).
6. **Tabela consolidada** — todos os experimentos lado a lado (a planilha) + **gráficos**.

7. **Melhor configuração final** — qual “pilha” venceu e o ganho total vs baseline (Δ por métrica).
8. **Análise crítica** — trade-offs (qualidade $\times$ latência $\times$ custo), o que NÃO funcionou e por quê, limitações do dataset.
9. **Conclusão** — aprendizados e próximos passos.
10. **Anexos** — prints de traces do LangFuse, exemplos de perguntas certas/erradas antes e depois.

9. Rubrica de avaliação (100 pts)

Critério	Pontos	O que se espera
Dataset de avaliação	15	Gabarito coerente, perguntas realistas, 3 tipos, sem viés de vocabulário
Rigor da medição	20	Experimentos controlados, baseline clara, métricas corretas (retrieval + RAGAS)
Cobertura de técnicas	20	Pelo menos: chunking, embedding, query enhancement, reranking + 1 avançada
Mudanças no código	15	Alterações corretas e bem localizadas no projeto final, versionadas
Análise e interpretação	20	Explica por que cada técnica ajudou/atrapalhou; discute trade-offs
Relatório (clareza, gráficos)	10	Organização, tabelas, gráficos legíveis, conclusão sustentada pelos dados

Destaques (bônus até +10): uso do LangFuse para diagnóstico; comparação OpenSearch $\times$ grafo em perguntas multi-hop; análise de custo/latência por técnica.

10. Armadilhas comuns (evite perder pontos)

- **Gabarito frouxo** → métricas “empatam” em tudo. Garanta relevância bem definida.
- **Pergunta que copia o texto** → baseline já acerta, nenhuma técnica “melhora”. Use linguagem do usuário.

- **Mudar duas coisas ao mesmo tempo** → você não sabe o que causou o efeito. Uma variável por experimento.
- **Esquecer de reindexar** após mudar chunking/embedding → você mede a config antiga.
- **RAGAS com Groq** → use strictness=1; fixe langchain-community==0.4.1 (Aula 5).
- **Modelo de reasoning (gpt-oss) para tarefas com tool/JSON** → instável; prefira llama-3.3-70b-versatile (ou equivalente OpenAI-compatible).
- **Comparar latência sem aquecer** → a 1ª chamada baixa pesos (Docling/embeddings); descarte o cold start.

11. Cronograma sugerido

Semana	Atividade
1	Escolher a fonte, ingerir no projeto, montar o dataset de avaliação, medir a baseline
2	Fases 1-3 (extração, chunking, embedding)
3	Fases 4-6 (híbrida/top_k, query enhancement, reranking)
4	Fase 7 (avançada) + Fase 8 (RAGAS)
5	Consolidar tabela/gráficos e escrever o relatório

12. Checklist final

- ☐ Fonte de dados própria ingerida no projeto
- ☐ avaliacao/dataset.json com perguntas + gabarito (+ resposta de referência)
- ☐ Baseline medida e registrada
- ☐ ≥ 5 experimentos (chunking, embedding, query enhancement, reranking + 1 avançada)
- ☐ Métricas de recuperação (Hit@K, Recall@K, MRR, NDCG@K) em todos
- ☐ RAGAS na baseline e na melhor config
- ☐ Planilha consolidada + gráficos
- ☐ Relatório com análise crítica e configuração final vencedora
- ☐ Código modificado e versionado por experimento